

# ELEC 378 Final Project Report

Building a Moth and Butterfly Classifier

Notebook and Model Checkpoints:  
[github.com/lijoannali/butterfly-moths-classifier](https://github.com/lijoannali/butterfly-moths-classifier)

## Team Information

Our team name is “I Love Butternut Squash Soup” and our names and NetIDs are: Cindy Nguyen (cn32), Ivana Zak (ivz1), Joanna Li (jl561), and Helena Wang (hw101).

### Team Member Contributions

**Cindy Nguyen** focused on Data exploration and Preprocessing and feature/design choices. Exploring the data included examining the images, class balance, visual variability, and any challenges in the dataset. Cindy also experimented with preprocessing steps such as resizing, normalization, cropping, color processing, augmentation, and feature extraction. Finally, Cindy contributed to Results interpretation based on the accuracy scores of the final model.

**Ivana Zak** explored the HOG descriptors and wrote about working with the HOG and kernel SVM descriptors. This involved implementing the initial version of the model and making changes to the implementation to improve its accuracy over the course of the project. Ivana also contributed sections relevant to the overfitting analysis of the kernel SVM model.

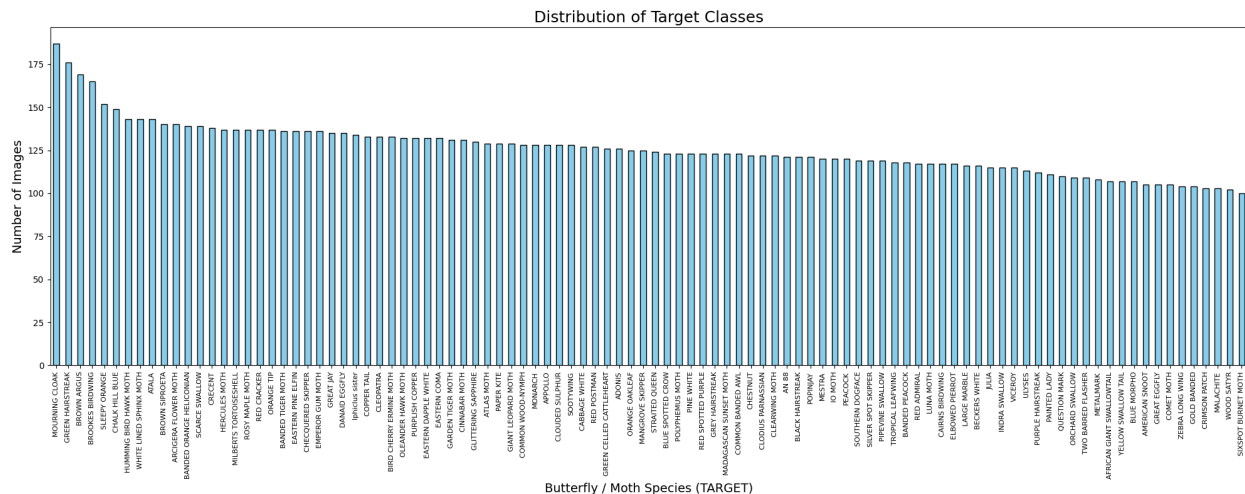
**Joanna Li** was responsible for working on the neural network model architecture and researching ways to train, add data augmentation, and write the code for prediction. This work also included documenting and researching ideas for improving model performance and recording validation loss over the training process to prevent overfitting. She also authored the sections explaining the design of our neural network model and architecture choices, final pipeline, created the ResNet and pipeline diagrams, and completed related work in the report.

**Helena Wang** was responsible for running and testing the model’s accuracy and tuning the neural network model and testing out improvements like dropout and a new scheduler; in addition, Helena contributed to the figures and report sections focused on the performance of the neural network.

## Data Exploration

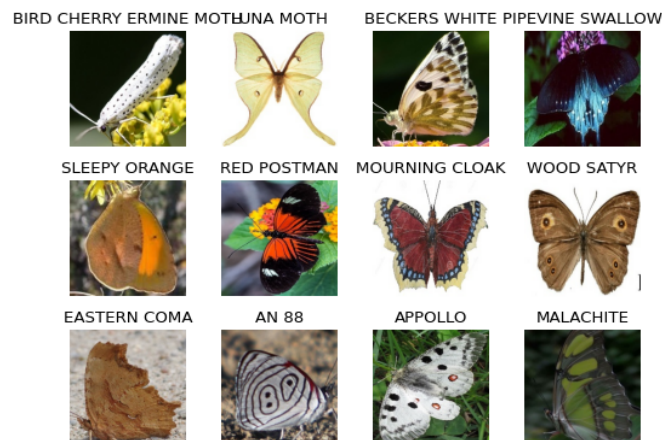
Before building the model, we performed data exploration to better understand the dataset and identify potential challenges. The dataset contains 12,594 labeled training images and 1000 unlabeled test images drawn from 100 butterfly and moth categories. While the number of images per class is fairly balanced, the images themselves vary significantly. They come in different sizes and aspect ratios, which means we need to standardize them. More importantly, the images show real-world challenges like heavy background clutter (leaves, branches, etc.), extreme pose variations (like wings fully open versus folded), and visually similar species that mimic each other's colors. Spotting these issues early was important. It showed that without the right data augmentation and cropping strategies, the model could easily get confused by background noise or struggle to tell similar species apart.

A closer look at the class distribution reveals that the dataset is exceptionally well-balanced for a multi-class problem. The vast majority of the 100 classes make up between 1.0% and 1.5% of the total data. Specifically, the most represented class, the Mourning Cloak, contains 187 samples (~1.5%), while the least represented class, the Sixspot Burnet Moth, contains 100 samples (~0.8%). On average, each class has about 126 samples, which is roughly 1.0% of the dataset. This confirms that no single class dominates the dataset. Such a balanced distribution is ideal for training a machine learning model, as it naturally prevents the classifier from developing a bias toward any specific species and ensures that the model learns to recognize all classes fairly.



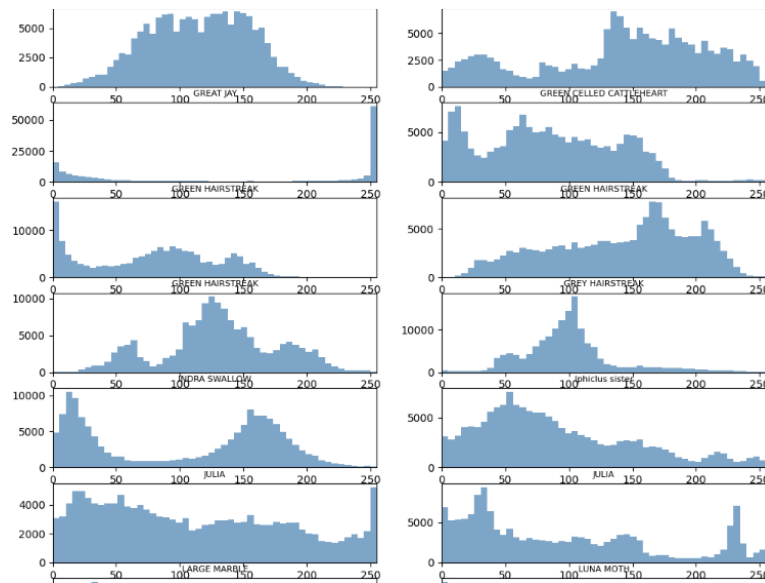
*The data was well-distributed among the 100 target classes*

To look at these challenges visually, we used a custom function called `explore_batch`. This function picks a random sample of 12 images from our data and displays them in a grid. This quick visual check was very helpful for getting a feel for the data. The batch results confirmed that the images vary wildly: lighting conditions, the size of the butterfly or moth inside the picture, and how busy the background is change drastically from image to image. This highlighted why we need a strong convolutional architecture that can handle spatial changes (spatial invariance).



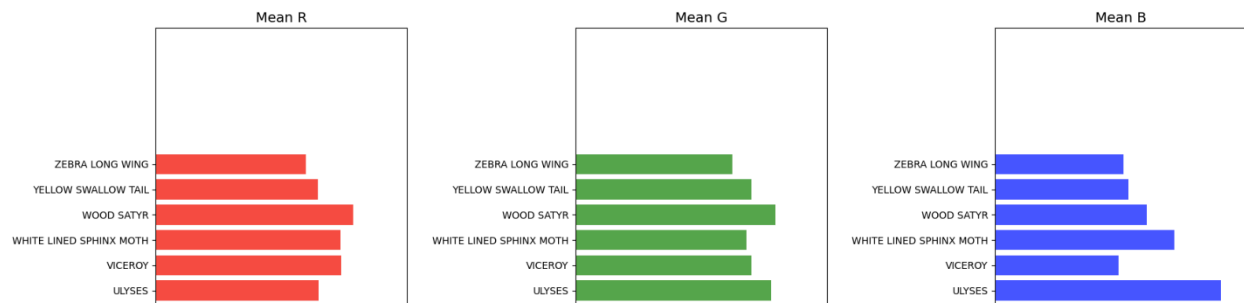
*Displaying a batch of random images*

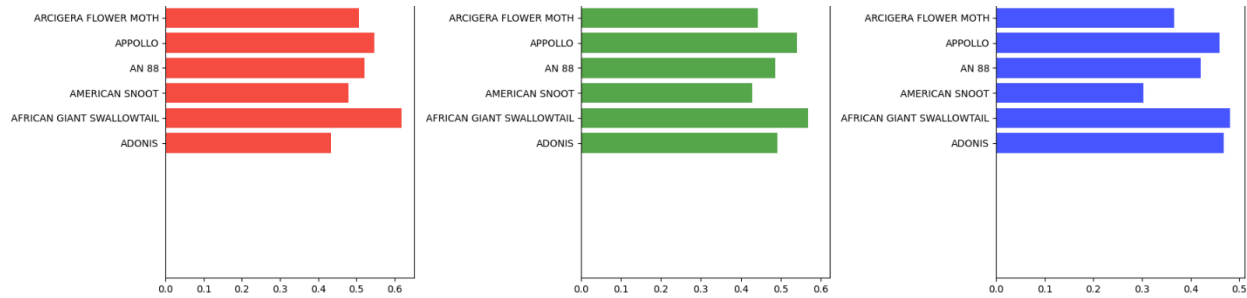
To go beyond just looking at the images, we used two functions to analyze the pixel data mathematically. First, the `explore_histograms` function sampled 100 images, sorted them by class, and plotted a histogram for each. These histograms show how the pixel intensities (from 0 to 255) are distributed. This helped us understand color variations and background noise. The plots showed a wide range of pixel intensities: some images had sharp peaks (meaning a plain, uniform background), while others had wide, flat shapes (meaning a busy, noisy background). Shown below are some of the resulting histograms (could not include all due to report length).



*Image intensity histograms*

Second, we used the `mean_rgb_per_class` function to calculate and graph the average Red, Green, and Blue (RGB) pixel colors for every class. This was a great tool to check for background color biases. It showed that certain classes were noticeably greener on average, strongly suggesting those species mostly had leafy, green backgrounds. This was a crucial warning: a simple model might accidentally learn to recognize the green leaves instead of the actual butterfly or moth. Shown below are the first few rows and last few rows of the bargraph (could not include the full bargraph due to report length).





Finally, we split the data into training and validation sets using a standard 80/20 split. We chose this because it is a proven standard in machine learning: it gives the model plenty of data (80%) to learn the deep features using the ResNet architecture, while keeping a completely unseen chunk of data (20%) to test how well the model generalizes and to check for overfitting. We also made sure to use class stratification during this split. Because we have 100 different classes, stratification was essential. It guarantees that the exact same ratio of each class is kept in both the training and validation sets. This stops the model from becoming biased toward classes that might have slightly more images.

## Preprocessing and Feature/design choices

### SVM with HOG model

#### 1. Preprocessing Steps

##### 1.1 Image Loading and Format Conversion

The raw image data undergoes a series of preprocessing steps designed to optimize the model's performance and ensure consistency across the dataset. First, the images are loaded into the pipeline using OpenCV, which by default reads image files into a BGR (Blue, Green, Red) color format. While this is standard for the library, it can cause discrepancies if later processing steps or visualization tools expect the more conventional RGB (Red, Green, Blue) format. Therefore, a conversion from BGR to standard RGB is performed using `cv2.cvtColor`. This ensures that the color channels are processed correctly in subsequent color-dependent feature extraction phases, such as the HSV color histogram calculation.

##### 1.2 Aspect-Ratio Preserving Resizing and Padding

The images must be standardized in size to be fed into the feature extraction algorithms, which expect fixed-size inputs. However, simply resizing images to a square dimension (256x256) often distorts the objects within them by stretching or squashing the content, which can severely degrade the quality of spatial features like HOG. To avoid this, a custom resizing function `resize_with_padding` is applied to fit the images to a

uniform 256x256 pixel dimension while preserving their original aspect ratio. The algorithm determines a scaling factor based on the largest dimension of the image, scaling it down or up until that maximum dimension reaches 256 pixels. To ensure the highest possible image quality during this transformation, the function dynamically selects the interpolation method based on the scaling direction. If the image is being scaled down (shrunk), it uses Area interpolation, which yields distortion-free results. If the image is being scaled up (enlarged), it uses Bicubic interpolation, which considers 4x4 pixel neighborhoods to create smoother, less pixelated edges. Once the image is resized, it typically has one dimension smaller than 256. The remaining space is then padded with black pixels (zeros) symmetrically, centering the resized original image on the new 256x256 canvas. This guarantees that all images have identical dimensions without any geometric distortion of the subjects.

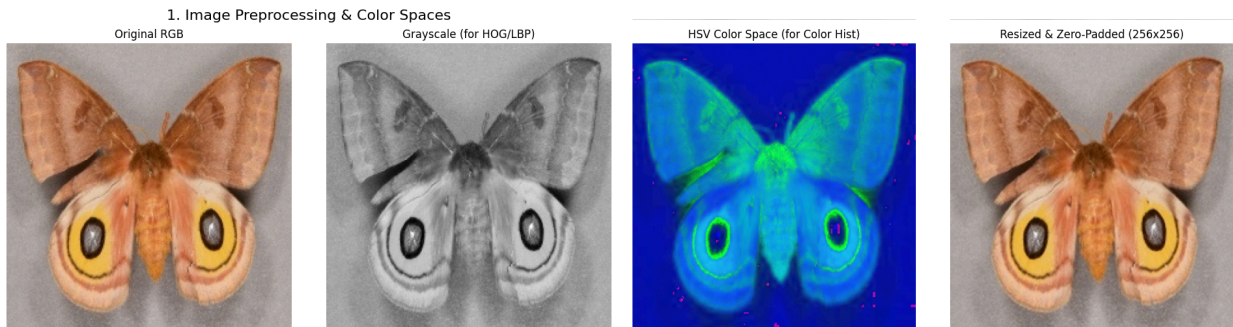
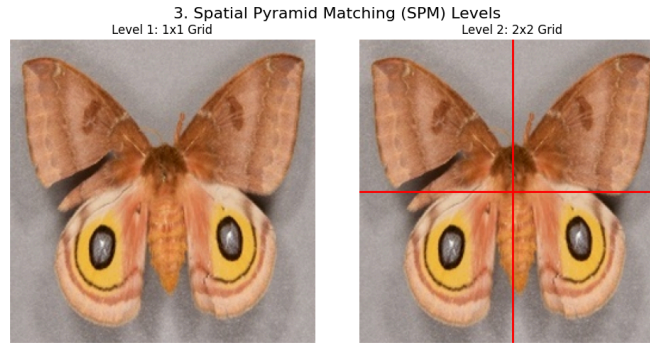


Figure: Image Preprocessing and Color Spaces on train\_006900.jpg

## 2. Feature Extractions

### 2.1 Spatial Pyramid Matching (SPM)

To capture both the broad, global characteristics of the images and the finer, localized details, the model relies on various hand-crafted feature descriptors extracted via a Spatial Pyramid Matching (SPM) scheme. The SPM is configured for two spatial levels: a 1x1 grid (Level 1) and a 2x2 grid (Level 2). At the 1x1 level, features are extracted globally from the entire 256x256 image, providing a macro-level summary of the image's contents. At the 2x2 level, the image is divided into four equal non-overlapping quadrants (each 128x128 pixels), and features are extracted from each quadrant independently. This multiscale approach yields a total of 5 spatial regions per image. By concatenating the features from all these regions, the model successfully retains important spatial context (for example, distinguishing between a texture that appears in the top-left versus bottom-right corner).



*Figure: SPM Levels on train\_006900.jpg*

## 2.2 Histogram of Oriented Gradients (HOG)

Within every localized patch defined by the SPM, the pipeline extracts three types of features to build a comprehensive representation and the first is HOG. The RGB patch is converted to grayscale beforehand, as HOG focuses purely on structural edge and shape information, independent of color. The HOG parameters are tuned to capture the structural nuances of the subjects. The orientations are set to 12, dividing the 180-degree gradient angle into finer bins than the default 9, allowing for more precise edge directionality. The `pixels_per_cell` is set to (16, 16), which controls the fundamental spatial resolution of the gradient histograms. To ensure robustness to local illumination and contrast variations, the `cells_per_block` is set to (2, 2), meaning the histograms are normalized over 32x32 pixel overlapping blocks. The normalization method chosen is 'L2-Hys' (L2-norm followed by clipping and renormalization), which limits the influence of very strong, isolated gradients. Also, a square root transform is applied prior to gradient computation to compress the dynamic range of pixel intensities, helping in shadow and illumination invariance.

## 2.3 3D Color Histograms

The second feature extracted from each patch focuses on the chromatic information. To robustly capture the color distribution regardless of minor lighting changes, the RGB patch is converted to the HSV (Hue, Saturation, Value) color space. HSV separates the color information (Hue and Saturation) from the brightness (Value), making the resulting histograms much more resilient to variations in lighting intensity. A 3D Color Histogram is then calculated across the three HSV channels. The histogram uses 12 bins for each channel (12x12x12), resulting in a 1728-dimensional vector per patch. The ranges are suitably set for OpenCV's HSV implementation: [0, 180] for Hue, and [0, 256] for Saturation and Value. Then, the 3D histogram is flattened into a 1D array. It is normalized by dividing by its sum (basically an L1 normalization), which turns the histogram into a probability distribution, representing the relative frequency of colors in the patch rather than absolute pixel counts.



## 2.4 Local Binary Patterns (LBP)

The final feature descriptor used in the patches is LBP, designed to map the micro-textures of the subjects, such as the intricate patterns on butterfly wings. Like HOG, LBP is computed on the grayscale version of the patch. The LBP extraction utilizes a circular neighborhood with a radius of 2 and `n_points` set to 16. This provides a slightly larger and more detailed receptive field than the standard 8-point radius-1 configuration. The ‘uniform’ method is employed, which groups all LBP codes with more than two spatial transitions (0-to-1 or 1-to-0) into a single "miscellaneous" bin. This drastically reduces the dimensionality of the LBP histogram from 65536 possible patterns to just  $n\_points + 2 = 18$  bins, while retaining fundamental textures like edges, corners, and flat regions. The resulting LBP histogram is also L1-normalized to represent texture frequency independent of patch size.

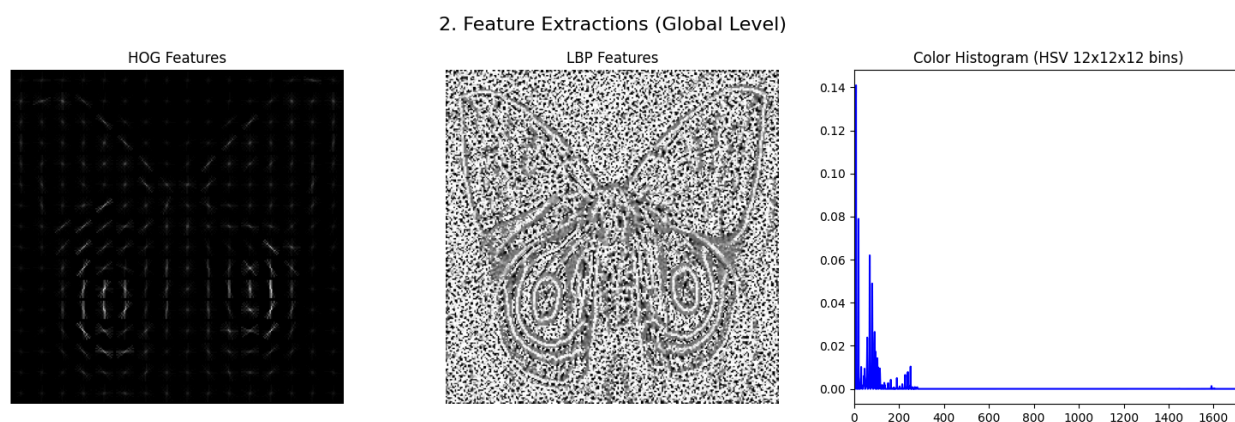


Figure: Edge and Pattern Feature Extractions by HOG, LBP, and Color Histogram on `train_006900.jpg`

## 3. Feature/Design Choices

### 3.1 Feature Concatenation and Initial Normalization

When the raw features are extracted for all spatial patches, they undergo specific design choices to normalize the data and prepare it for the classification algorithm. First, all the HOG features from the 5 patches are concatenated together, as are the Color and LBP features. Before combining the three modalities into one descriptor, each concatenated feature type (global HOG, global Color, global LBP) is independently L2-normalized. This ensures that the vector length for each feature type is 1. Independent normalization is critical because HOG, Color Histograms, and LBP inherently operate on different scales and represent completely different statistics. By L2-normalizing them, they are brought to a comparable numerical scale before they are concatenated into a unified feature vector representing the entire image.

### 3.2 L2-Power Normalization

To further refine these unified features and mitigate specific statistical artifacts commonly found in visual words and dense descriptors, the power normalization strategy L2-Power is used with an alpha parameter of 0.5. Power normalization works by



taking the signed square root ( $\text{sign}(x) * |x|^{0.5}$ ) of every element in the feature vector. In the context of visual features, this operation is effective at suppressing very high values (burstiness), which occurs when a specific visual element (like a repetitive texture) appears many times in an image, dominating the feature vector and overshadowing other important but less frequent features. By taking the square root, the distribution of feature values becomes more uniform. Following this element-wise square root transformation, the entire vector undergoes another round of standard L2 normalization to ensure the final features reside on a unit hypersphere, which is optimal for distance-based classifiers like SVMs.

### 3.3 Principal Component Analysis (PCA)

Following the normalization, the data is standardized across the entire training dataset to have a zero mean and unit variance for every feature dimension using Scikit-Learn's StandardScaler. Standardization is a prerequisite for many machine learning algorithms, including SVMs and PCA, as it ensures that features with larger numerical ranges do not artificially dominate the objective function. Since this combination of SPM, HOG, LBP, and Color Histograms creates a massively high-dimensional space, PCA is configured to project the standardized data down to 500 essential principal components. This dimensionality reduction serves multiple purposes: it significantly reduces computational overhead for the classifier, mitigates the curse of dimensionality, and acts as noise reduction by removing variance associated with less significant visual artifacts. The same PCA transformation learned on the training set is applied to the validation and test sets.

## VGGNet + ResNet V2 model

### 1. Preprocessing Steps

#### 1.1 Image Loading and Color Format Standardization

The initial data preprocessing and loading strategy focuses on establishing a robust and reliable baseline for neural network training. First, the images are loaded and converted into a standard RGB format. This conversion step is critical to ensure color consistency across the dataset, mitigating any potential issues from grayscale images or alternative color channels. Then, the images are initially resized to a uniform 224x224 dimension, which is standard in using architectures inspired by ImageNet, as it strikes an optimal balance between preserving fine-grained spatial hierarchies and maintaining computational efficiency.

#### 1.2 Categorical Label Encoding

To prepare the textual and string-based categorical labels for the neural network, a robust preprocessing step utilizing Scikit-Learn's LabelEncoder is employed. The neural network inherently relies on numerical tensors to compute gradients and loss functions. Therefore, the LabelEncoder systematically maps the string-based class names

into a structured integer format ranging from 0 to 99, corresponding to the 100 distinct target classes. This transformation ensures the classification target is compatible with the standard Cross-Entropy Loss optimization algorithm.

### 1.3 Dataset Partitioning and Stratification

For the structural setup of the training phase, the dataset undergoes a train/validation split process using a `test_size=0.0079`. This reserves 100 images for validation, allowing us to train the model on the maximum number of training images as possible (12,494). Crucially, this split is executed with stratification enabled. Stratification forcefully ensures that the exact original distribution of the 100 classes is perfectly preserved in both the training and validation subsets, thereby preventing the model from becoming biased toward statistically overrepresented classes. Furthermore, to guarantee absolute reproducibility, a `random_state` parameter is seeded with the value 347. In addition to the standard holdout, a 5-fold cross-validation scheme is integrated to evaluate the model across non-overlapping partitions, guaranteeing robust generalization to unseen data.

### 1.4 Data Augmentation for Training

Different synthetic variances are introduced into the training images, inflating the dataset size and compelling the model to become robust to real-world visual distortions. The `train_transform()` pipeline leverages PyTorch's transformations to sequentially apply geometric augmentations: a `RandomHorizontalFlip` ( $p=0.5$ ), a `RandomVerticalFlip` ( $p=0.3$ ), and a `RandomRotation` with a maximum degree range of 30. These reflections and rotations ensure that the network does not falsely learn directional dependencies. Profound visual and textural variations are injected via `ColorJitter` (shifting brightness and contrast by up to 30%). The pipeline employs `RandomResizedCrop(224, scale=(0.8, 1.0))`, which extracts a localized patch of the image and dynamically resizes it back to 224x224. A `RandomGrayscale` transform ( $p=0.1$ ) forces the network to occasionally rely purely on structural geometry rather than colorimetry, while `RandomErasing` ( $p=0.2$ ) actively masks out randomized rectangular patches, acting as spatial dropout that forces the network to infer classifications even when critical features are obscured.

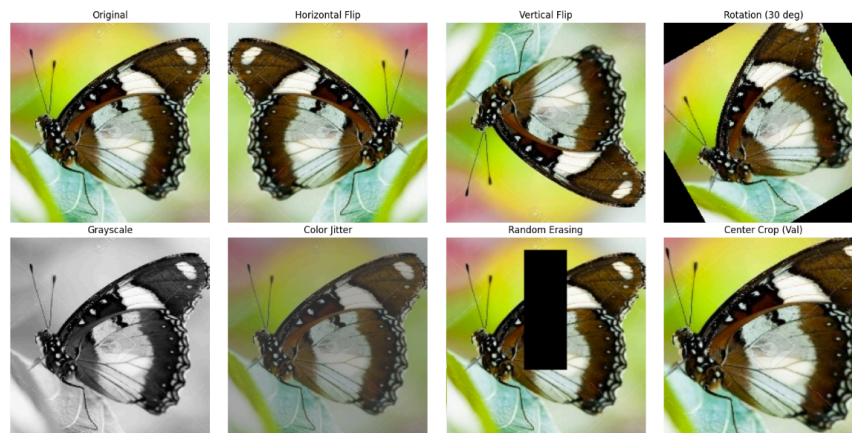


Figure: Various types of Transformations applied to sample image `X_train_files[4]`

## 1.5 Validation Set Transformations and Normalization

The preprocessing pipeline applied to the validation set (`val_transform`) incorporates a `CenterCrop(IMG_SIZE - 40)` operation, which excises 20 pixels from all four borders of the image (reducing it to 184x184) before a high-quality interpolation resize back to the standard 224x224 resolution. This architectural choice mitigates the overwhelming statistical influence of noisy background colors and harsh edge artifacts. Finally, both the augmented training tensors and the cropped validation tensors are normalized using ImageNet mathematical constants ( $\text{mean}=[0.485, 0.456, 0.406]$ ,  $\text{std}=[0.229, 0.224, 0.225]$ ), which centers and scales the tensor distributions to stabilize gradient propagation.

## 2. Feature Extractions

### 2.1 Implicit Feature Extraction via Tensor Conversion

Contrary to traditional machine learning pipelines like SVM that rely on hand-crafted feature extractors (such as HOG and color histograms), this method relies on the deep neural network to perform automated, hierarchical feature extraction. Therefore, the feature extraction phase during preprocessing is entirely focused on providing the cleanest possible raw pixel data to the network. The images are converted from raw JPEG format into the mathematical representation of PyTorch tensors (formatted as Channels  $\times$  Height  $\times$  Width), ensuring that the underlying spatial relationships and pixel intensities are preserved for the convolutional layers to parse.

### 2.2 Normalization for Feature Scaling

To optimize the automated feature extraction process, the pixel values are scaled and normalized using standard ImageNet constants ( $\text{mean}=[0.485, 0.456, 0.406]$ ,  $\text{std}=[0.229, 0.224, 0.225]$ ). By centering the pixel distributions around zero and scaling their variances, this preprocessing step ensures that the network's feature extractors (the convolutional filters) are not biased by extreme lighting conditions or abnormal color saturation in the raw images.

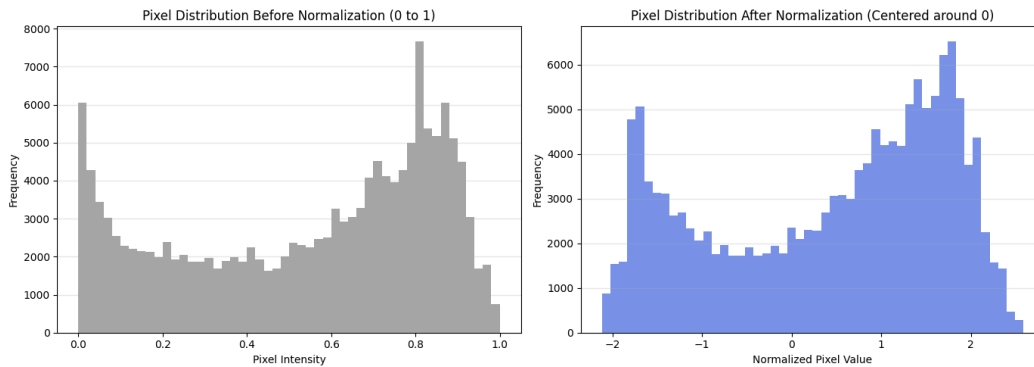


Figure: Pixel Distribution plots Before vs. After Normalization for sample image `X_train_files[4]`

### 3. Feature/Design Choices

#### 3.1 Standardized Spatial Resolution

The decision to resize all images to a uniform 224x224 resolution is a deliberate design choice that balances computational efficiency with spatial feature complexity. This specific resolution is the standard for convolutional architectures, providing a large enough grid to capture intricate anatomical features of the insects while preventing the network from exceeding memory constraints during batch processing.

#### 3.2 Augmentation for Feature Robustness

The inclusion of specific augmentations like RandomErasing and ColorJitter was a design choice to prevent the model from memorizing superficial features. By randomly dropping out sections of the image (erasing), the network is forced to learn holistic, structural features rather than relying on a single defining characteristic (such as a specific wing pattern). Similarly, color jittering ensures the model focuses on morphology and geometry rather than purely memorizing lighting and color conditions.



Figure: Variations with multiple random transformations applied to sample image `X_train_files[4]`

#### 3.3 Stratification and Cross-Validation for Generalization

Rather than relying on a simple randomized split, the design choice to enforce label stratification ensures that the statistical distribution of all 100 classes remains uniform across the training and validation sets. Along with the implementation of a 5-fold cross-validation scheme, these data partitioning choices guarantee that the model's performance metrics are a true reflection of its ability to generalize to unseen data, mitigating the risk of class imbalance skewing the results.

## Models considered

### Kernel SVM

For our first model, we decided to try Kernel Support Vector Machines (SVMs) as they are one of the most simple and cost-effective linear classification algorithms, especially when compared to more complex models like neural networks. We decided to use a kernel SVM specifically (as opposed to a regular SVM) because it was very likely that the reduced - dimensional feature vectors from HOG + color histogram + LBP were not linearly separable in their original dimensional space.

The main idea of a binary kernel SVM is to draw a hyperplane through the data, using the formula  $f(x) = \langle w, x \rangle + b = 0$  such that the data on one side of the hyperplane ( $f(x) > 0$ ) is one class while the data on the other side ( $f(x) < 0$ ) is the other class. Therefore, we can predict the label of any data point using the formula that  $= \text{sgn}(\langle w, x \rangle + b)$ . In order to find the most stable hyperplane, we introduce the concept of a margin. The margin is the distance between the two hyperplanes  $\langle w, x \rangle + b = -1$  and  $\langle w, x \rangle + b = 1$ , together called the “support vectors”. Thus, the margin always has width  $2/||w||$ . In the linearly separable case, the support vectors are normalized such that they intersect the closest data point to the hyperplane  $f(x)$  from each respective class. However, it is most often the case that the data is not completely linearly separable, breaking down our original definition of the support vectors. To address this issue, we allow the idea of a “soft” margin instead, where data points can be both inside the margin and on the opposite side of  $f(x)$ .

However, we must introduce another metric that quantifies both the error of a misclassified data point and the error of a data point that violates the margin. This metric is called the hinge loss, and is defined as:

$$\ell(y_i, f(x_i)) = \max(0, 1 - y_i f(x_i))$$

*Hinge loss*

This formula gives a 0 error for correctly classified points, an error between 0 - 1 for points that violate the margin, and an error that increases linearly beyond 1 for misclassified points. Therefore, our goal now becomes to find  $w$  and  $b$  such that we maximize the margin while minimizing the hinge loss. This can be mathematically defined as having a loss function defined as:

$$\mathcal{L}(w) = \lambda ||w||^2 + \frac{1}{N} \sum_{i=1}^N \max(0, 1 - y_i(w \cdot x_i + b))$$

*SVM objective with hinge loss and maximum margin objective*

where  $\lambda$  is a “tuning parameter” - a larger  $\lambda$  places more emphasis on minimizing the weights where a smaller  $\lambda$  places more emphasis on minimizing the hinge loss.

There were a few remaining issues we had to address before we could run the SVM on our data, however. For one, it was assumed that our feature vectors were likely not linearly separable in their original dimensional space, meaning that we had to use a nonlinear transformation on our data matrix first in order to transform our data to a higher dimensional space where it would be linearly separable. We decided to use a radial basis kernel / transform with the hyperparameter “C” = 10 (C is the inverse of  $\lambda$  above) and the hyperparameter “gamma” set to “scale” (explained in the Training & Tuning section) The last major remaining issue was that SVM is inherently only a binary classifier, whereas we needed a multiclass classifier. The approach we decided to take was “one - vs - one” (OvO) approach (as explained in the Training & Tuning section), where each class is tested against every other class one at a time and the label of a data point is determined by the label that was chosen most often.

## Neural Nets

After testing out the HOG-based descriptors, we looked into convolutional neural networks (CNNs) because they are known to perform well on image classification tasks. On a high level, the goal of the CNN model is to learn filter weights for kernels at different layers that we can use to process the raw pixels of an input image down into representations called feature maps, which are eventually flattened to a linear layer that we run softmax on to assign a class to each output.

## How CNNs Differ from Fully Connected Deep Neural Networks

Compared to a fully-connected neural network, a CNN's layers do not connect every input node to every output node. A CNN looks over each patch of the input image, and reduces the raw pixels to a feature map, which measures how much each patch responds to a filter. In many CNNs, the filters in earlier layers often look like edge detectors, and progress to detect more and more detailed features in deeper layers closer to the output. Instead, each input layer is processed with a convolution operation where a kernel is slid over a small patch of the input layer, and the result of the convolution is given to one output node. The weights in the filters are not handcrafted, but also learned through backpropagation and training the network.

## Pooling Layers

Compared to the HOG-based classifier, our CNN-based models use backpropagation to learn kernel weights, which leave the problem of learning scale invariant features. To achieve this, MaxPool layers are based to lower the dimensions of the feature maps so that the network can learn features that are informative at different scales.

Rather than storing different weights for the connection of each input node to each output, only the weights in this shared kernel need to be learned and stored for each CNN layer. This reduces the number of weights and biases that must be learned, and means that CNNs use the local context around each pixel in the input layer– and uses its relationship to the local pixels around it to measure a response from the convolution filter and then passed to the output.

## Residual Networks and the ResNet V2 Block

Our final neural network model consisted of ResNet V2 models composed into an architecture inspired by the VGGNet paper. The model used a negative cross-entropy loss objective, which optimizes for having the correct class receive the highest predicted probability. For each training example the model produces a raw score (logit) for every class, which is then converted to probabilities via softmax.

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log \left( \frac{e^{x_{i,y_i}}}{\sum_{c=1}^C e^{x_{ic}}} \right)$$

*Cross entropy loss function for classification*

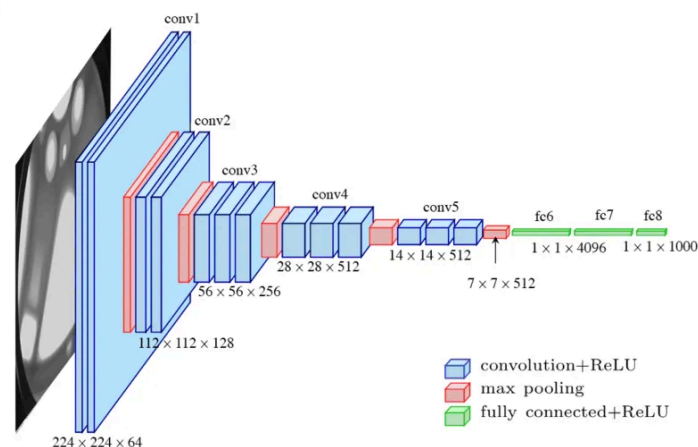
The loss then takes the log of the probability assigned to the true class and averages this quantity (negated) across the batch. Intuitively, the model is penalized heavily when it assigns low confidence to the right answer, and rewarded when it concentrates probability mass on the correct class.

Hyperparameters such as the batch size and learning rate were initialized to 64 and  $1e-4$  because those values worked from experience in previous image classification models Joanna has built. Over time, we did not adjust the batch size because it produced accurate results, but we changed between  $1e-3$  and  $1e-4$  for the learning rate without finding significant differences in training performance or test accuracy.

This model used the VGGNet structure and residual network skip connections that were discussed in class, but expanded beyond those connections with the use of ResNet V2 models that use batch normalization.

## VGGNet

The VGGNet can be visualized as a funnel shape that uses CNN blocks to change the input  $224 \times 224$  image into representations with smaller dimensions and more channels after each step, before softmax and a linear layer are used at the end to assign a class to the output <sup>1</sup>. The side length of each representation is halved at each stage with a max pooling block while the convolution filter bank adds more kernels so that the output is returned with more and more channels in the deeper layers of the network.



*The VGGNet architecture which gradually downsamples the feature map while doubling the number of channels with increasing depth*

The two main benefits of this architecture are that the model can learn features that are invariant to translation and scaling, while retaining the learning it does in earlier layers even as we move to the deeper layers. As each pooling stage compresses the image into smaller and smaller spatial dimensions—the receptive field, or the “learning context source” of each node that exists in layers deeper down the neural network grows larger and larger. As you go deep into the VGGNet architecture, we keep the size of the  $3 \times 3$  kernel constant but apply it to smaller

<sup>1</sup> VGGNet: <https://arxiv.org/pdf/1409.1556>

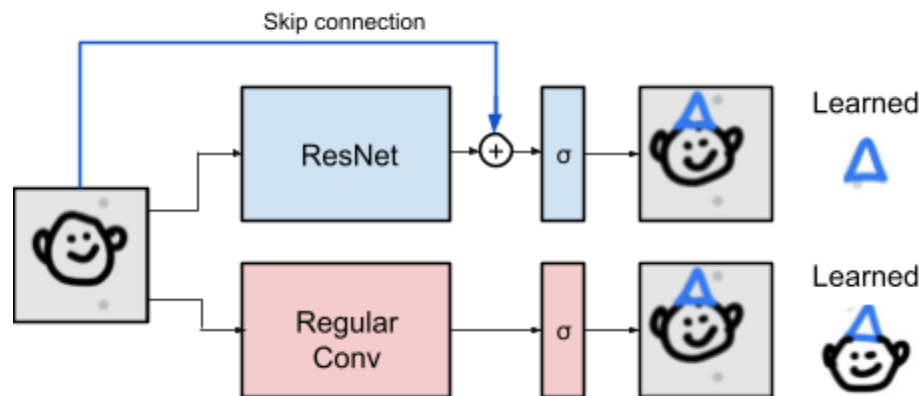


and smaller downsampled feature maps that represent the original 224 x 224 image, it is as if you are covering more and more of the original image with each step. By the end, on the 14 x 14 representation, the 3x3 kernel covers about a fifth of the whole image.

Following the innovations of the original VGGNet, we used a smaller filter size with a 3x3 kernel on the first layers of the network– and this smaller convolutional kernel size was a choice that allowed us to keep the number of weights small even as we increased the depth of the model. Like the original VGGNet, we also used the ReLU nonlinearity after each convolutional layer. However, we also chose to make a change during development to use the ResNet V2 block to add skip connections inside of using regular CNN blocks, because we achieved lower validation loss within less epochs with the use of batch normalization.

## ResNet

When it became widely known that adding more layers to neural networks to add complexity with greater depth instead of width improved performance, an associated issue that arose was the problem of disappearing gradients and of the degradation problem– in which accuracy will saturate and fall after a certain network depth with no more consistent improvement with added depth. ResNet was a model that introduced the use of skip connections to solve this problem in deep networks.



*Compared to a regular CNN block, the skip connection in ResNet makes it so the model is only responsible for learning the residual, or difference between the input and the output.*

In a normal CNN block, the input is roughly processed by weight layers into an output and passed through a final activation function. Compared to this, a residual block adds its input back to the output right before the final activation function layer. This makes it so that the layer's weights need to only learn the difference between the input and output to the layer, instead of learning weights that build up the output from scratch<sup>2</sup>. Through our experimentation, we found that our model converged and reduced validation loss faster through the use of ResNet blocks compared to regular CNN blocks that we explored very early on. As we will discuss further in the training procedures section, we achieved validation losses with cross entropy scores of around

---

<sup>2</sup> [https://d2l.ai/chapter\\_convolutional-modern/resnet.html](https://d2l.ai/chapter_convolutional-modern/resnet.html)

1.0 at each 40 epochs, while previous models (not discussed or used in the final submission) we built used regular CNN blocks and only reached 1.5 loss after 100 epochs of training.

## **ResNet V2**

After finding that our accuracy on the public leaderboard rose from 72% to 87% by changing our regular CNN blocks to the original ResNet block, we explored the option of using ResNet V2 to further boost our accuracy.

Compared to ResNet, ResNet V2 applies batch normalization and ReLU activation before each convolutional layer twice within the block, so compared to other ResNet variants it performs a full pre-activation<sup>3</sup>. The feature maps going in as input to each ResNet block are passed through these two activation functions which push negative values to zero and then renormalized by subtracting their mean and dividing by their variance— roughly— so that weights in different layers which drift far away from each other over the course of training are brought back to a common standard. This was found in research to allow models to converge during training with less time and using greater learning rates compared to training without batch normalization, so we added this to our implementation<sup>4</sup>.

## **Training procedure and tuning**

### **Training the Kernel SVM**

Training and tuning the SVM was relatively simple. In order to train our SVM, the first step was to split our training data into training and validation data. We did this by using the `train_test_split` function in `sklearn` with the `test_size` as 0.2, random state 4242, and stratifying based on class distribution in order to preserve class distribution in both the training and validation data. Additionally, we chose an 80 / 20 split simply because it is a common way to split training and validation data. Next, we had to choose the SVM parameters: the kernel, the hyperparameter values “C” and “gamma”, and whether to use an One vs Rest (OvR) approach or One vs One (OvO) approach for multiclass classification. First, we chose OvO as opposed to OvR since OvO usually performs better when there are similar or overlapping classes. Our classes were considered similar as every class was a different variation of butterfly or moth, which are very similar to each other in most aspects except for coloring. Second, for a given C and gamma (before optimization), we tried a radial basis kernel and a polynomial kernel since we had learned about those two kernel choices in class. However, after running both and manually changing the degree of the polynomial kernel to several different values, it seemed that the rbf kernel always outperformed (in terms of classification accuracy) any polynomial kernel that we used. Therefore, we decided to stick to an rbf kernel and optimize for C and gamma. In order to do this, we used the function `GridSearchCV` with C values [1, 10, 100], gamma values ['scale', 1e-4, 1e-3, 1e-2], `cv = 2`, and `scoring = “accuracy”`. The C and gamma values for the grid search were chosen to cover a large range of possibilities for C and gamma while also not being too long of a list so that the model did not take forever to run. The decision to use `cv = 2`, where the function does k-folds cross validation using `k = 2` for the training data for each iteration of the

---

<sup>3</sup> <https://medium.com/analytics-vidhya/deep-dive-into-resnets-eb4ec48dfcb0>

<sup>4</sup> [https://d2l.ai/chapter\\_convolutional-modern/batch-norm.html](https://d2l.ai/chapter_convolutional-modern/batch-norm.html)

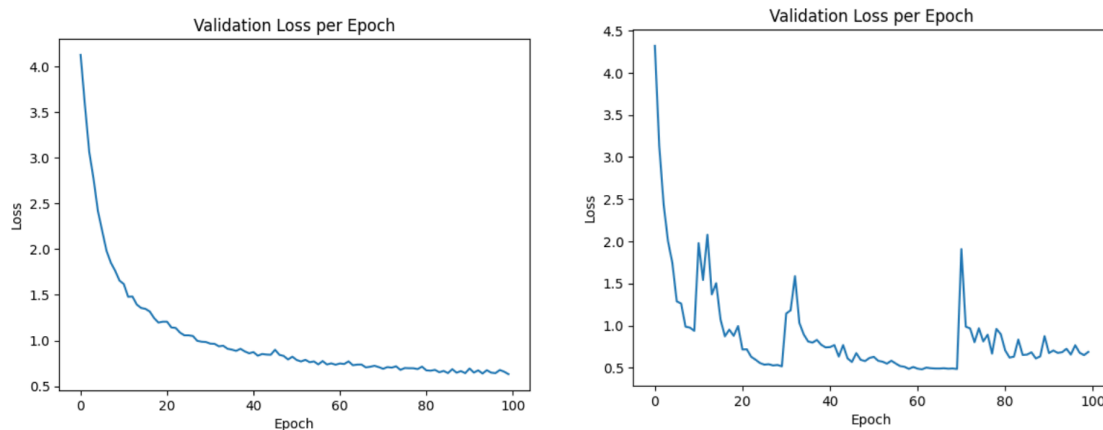
grid search, was also chosen for runtime considerations. Through this grid search, the best hyperparameters were found to be  $C = 10$  and  $\gamma = \text{"scale"}$ .

## Optimization Strategy for the Neural Networks

During training, the model was optimized using the Adam (Adaptive Moment Estimation) optimizer, with a baseline learning rate of  $1e-3$ . Adam was chosen because it computes individual adaptive learning rates for every parameter based on estimates of first and second moments of the gradients, which generally leads to faster and more stable convergence compared to standard stochastic gradient descent. This optimizer is dynamically governed by a CosineAnnealingWarmRestarts learning rate scheduler, with an initial restart period of  $T_0=10$  epochs, a restart period multiplier of  $T_{\text{mult}}=2$ , and a minimum learning rate of  $\eta_{\text{min}}=1e-6$ . This schedule gradually decays the learning rate following a cosine curve, then periodically resets it to help the model escape shallow local minima, with each successive restart period being twice as long as the previous. These oscillatory, warm-restarts help the optimizer escape shallow local minima and improve exploration of the parameter hyper-dimensional loss landscape, allowing for progressively finer and smoother optimization in later training stages.

Batch normalization was incorporated in a pre-activation arrangement (BN→ReLU→Convolution) within each residual block, where `nn.BatchNorm2d` normalizes the feature maps of each channel during training. This stabilizes optimization, reduces internal covariate shift, and allows for faster convergence. Placing batch normalization before the convolution layers also keeps the residual skip connection path unaltered, preserving clean gradient flow during backpropagation across the four residual blocks.

During training, checkpoints were saved at selected epochs containing the model weights, epoch number, and validation-loss history. After training, the checkpoint with the lowest observed validation loss was selected and loaded to generate final predictions on the test set.



*The validation loss was recorded after the training of each epoch during the training for the ResNet V2 and ResNet V2 with VGGNet structure models. After training, the epoch with the lowest validation loss was selected for prediction*

# Quantitative comparison of models

Based on the Kaggle submission results, our VGGNet + ResNet V2 inspired model performed the best with an accuracy of 95% on the public leaderboard and 95.8% on the private leaderboard. Compared to this, the ResNet V2 model didn't structure the dimension of its layers according to the VGGNet structure and achieved an accuracy of 94.8%.

Looking at the validation loss curves, the ResNet V2 model converged smoothly and monotonically from to ~0.50 over 100 epochs, with its lowest validation loss occurring at the final epoch 100, suggesting the model had not yet fully converged and may have continued to improve with additional training. In contrast, the VGGNet + ResNet V2 model reached a lower minimum validation loss of 0.4813 at epoch 62, indicating more efficient convergence, despite exhibiting periodic loss spikes attributable to the warm restart cycles of the learning rate scheduler. These validation loss curves, informed by 5-fold cross-validation, were used to identify the optimal checkpoint epoch (selecting the point of lowest validation loss before overfitting emerged) and also visually confirmed the effect of the CosineAnnealingWarmRestarts scheduler in the VGGNet + ResNet V2 model, with the periodic spikes and recoveries corresponding to the warm restart cycles contributing to better generalization and faster convergence.

As opposed to the neural network models, the Radial Basis Function kernel SVM performed significantly worse. The highest accuracy we were able to achieve with the SVM model was 0.646 and 0.612 on the public and private set, respectively, as opposed to 0.950 and 0.958 using the neural network. However, in terms of computational cost, our SVM was much lower. This is evidenced by the fact that it took significantly less time to run the model (about 40 minutes) compared to the final iteration of the neural network (3 hours). For the SVM, our training accuracy was 100% while our validation accuracy was 56.25%. However, because we analyzed the performance of each model using different metrics, it is difficult to equivalently compare the two models on other metrics besides overall test accuracy and computational cost. Overall, the neural network was able to more accurately predict the correct labels of each butterfly and moth species as it was able to learn a unique, nonlinear data transform that could then better linearly separate the data as opposed to the kernel SVM. On the other hand, the kernel SVM took significantly less time to train, making it a better model in terms of computational cost.

## Overfitting and generalization analysis

### **Overfitting in the Neural Nets**

For the neural networks, we avoided overfitting by early stopping. We tracked the validation loss on a held-out validation data set as we trained after each epoch, and we only saved the current model if its validation loss was lower than the previous minimum loss. This meant that we stopped saving the models when the validation loss stopped improving and started rising in the later epochs– which is indicative of overfitting. This meant that in practice on the final model we saved the model at epoch 61 instead of the final one at 100 because in later epochs the validation loss started rising again.

As discussed previously, we also implemented data augmentation with data transforms to randomly change alter images to increase the diversity of the training data and also improve generalization by learning rotation and scale invariant features. There wasn't any significant gap between the test performance and the performance on the public leaderboard to discuss, as both performed with similar high level levels around 95%.

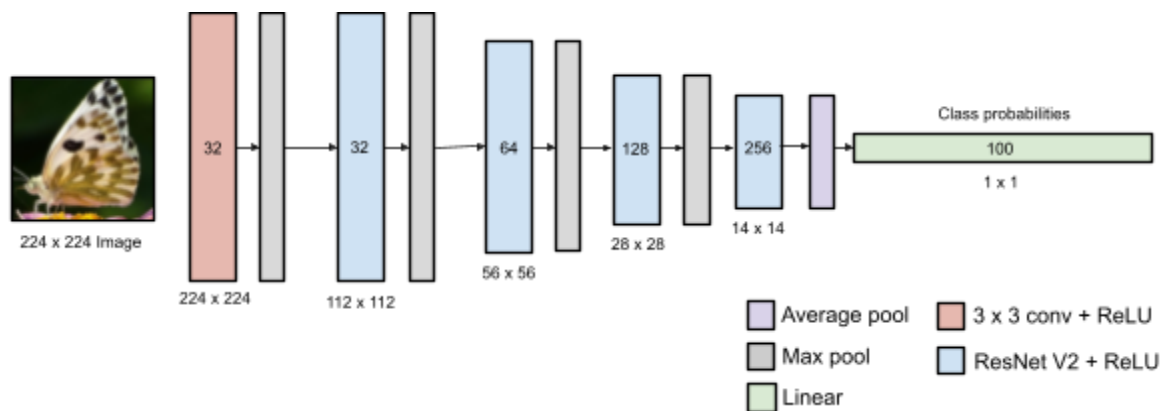
### **Overfitting in the Kernel SVM**

The main evidence of overfitting for our kernel SVM model is that our training accuracy was 100%, while the validation accuracy was only 56.25%. This strongly suggests that the model practically memorized the entire training set, including the noise present in each picture. This is not surprising, however, given several aspects of our model set up and overall classification problem. For one, we used a Radial Basis Function kernel, which is able to produce a very nonlinear boundary between classes. Although this can be a positive aspect when a very nonlinear boundary is needed to separate classes, this also means that the model can bend around individual data points / clusters in ways that do not generalize well to unseen data. On top of this issue, we also used a relatively high regularization hyperparameter, C, which placed more emphasis on minimizing the hinge loss as opposed to the margin width. Minimizing the hinge loss as opposed to the margin width again means that the model will have very few classification errors, but the stability / generalization of the model will be low, leading again to an overfitting issue. Third, because our classification problem is very complex, with different species still having very similar features, it's kind of inherent in the problem that the model is going to learn a very intricate, nonlinear decision boundary that is not robust to changes in order to learn the correct boundary for classes that seem very alike. Although it would seem like we could improve the generalization of our model through picking a simpler kernel or reducing the size of our hyperparameter C, we found when testing and tuning the model using different kernels and values of C that the kernel and hyperparameter value we chose were still the ones that had the highest validation accuracy. Additionally, when we tested this model on the actual test data, our public score was 64.6% accuracy, which is much higher than our 56.25% validation accuracy. Both of these indicate that a complex decision boundary is needed in order to correctly classify each moth and butterfly species, and that our model still generalizes fairly accurately. Additionally, the fact that the accuracy for the test data was higher than our validation accuracy indicates that perhaps the train / test split did not accurately represent the overall data distribution.

## **Final submitted pipeline**

The preprocessing for the neural network started with loading in the images with data augmentations. The purpose of the augmentations was to increase the size of the training data by adding modified versions of the butterflies and moths which were randomly rotated, flipped, had their colors randomly shifted, randomly cropped, or altered with black boxes. This also served the purpose of creating new labeled pairs that the model can use to learn features that can, say, detect an Adonis butterfly as Adonis despite any small variations in the blue pixel intensities or the image placing the butterfly on the left instead of the right side of the image.

For the neural network approach, we chose to use a VGGNet model with skip connection layers that used the ResNet V2 architecture and we trained for a maximum of 100 epochs with a batch size of 64 images and the ADAM optimizer. The learning rate was set to 0.001 and changed over the training epochs with a cosine annealing scheduler using warm restarts. The Adam optimizer was used to take advantage of momentum for faster convergence to a better fitting model while performing stochastic gradient descent. We used a cross validation and validation loss tracking process described in the relevant section of this report to select the best model to use for prediction, and save the parameters of the selected model



*Final submitted pipeline*

For prediction, we loaded the model we saved with the lowest validation loss from the training process, used the test dataloader to retrieve our data, and then put the model in eval mode so that the behaviors of stochastic features like batch norm and dropout will be made deterministic for creating the prediction. From here, we extract the outputs of the final layer to extract the class assignment for each data point and export this to a CSV file for submission.

## Results interpretation and conclusions

### ResNet V2 and VGGNet Model

Our final model achieved a strong accuracy of 95.0% on the public test dataset and 95.8% on the unseen private dataset. The fact that it performed slightly better on the private data shows that the model successfully learned the real underlying patterns of the images, rather than just memorizing the training set. This good performance was driven by a few key choices in our pipeline. First, by heavily augmenting our data (randomly flipping, cropping, altering colors, and erasing parts of the training images), we forced the model to become robust against real-world noise. Second, using a ResNet-style architecture with shortcut connections allowed us to build a deeper network to learn complex image features without the training process breaking down with vanishing gradients or degradation that led to poorer accuracy or worse generalization to the test data in previous iterations of our model.

I believe the final method performed well because the ResNet V2 skip connections and batch normalization helped the model learn more descriptive and distinguishing features from the dataset, while the optimizations we made for the learning rate scheduler helped the training process escape local minima and better explore the feature space of model weights and biases and converge on a well performing set of them. The parts of the pipeline that mattered the most to performance were the ResNet blocks and the VGGNet-inspired architecture. When we changed from a plain CNN model with the same architecture described above to the ResNet blocks, our accuracy on the public leaderboard changed from 72% to 87%, and after implementing the VGGNet structure that halved the channel dimension and doubled the number of channels at each stage, we were able to achieve accuracies above 90%. After this, further optimizations were achieved through changes to the learning rate scheduler and tuning the batch size.

### **HOG and SVM Model**

In terms of the SVM model, it was interesting to learn that most of the work creating the SVM model was spent creating the feature vectors that the model could use in the first place. Thus, one major step we would do differently if trying to build an SVM model would be to create better feature vectors. If we were to create feature vectors that focused on features that were most dissimilar across different butterfly and moth species, the original feature space would be closer to being more linearly separable in the first place. Another aspect we could try would be to test different kernels. We only tried rbf and polynomial kernels, but it could be the case that a sigmoid or an uncommon kernel type would allow the model to generalize better. Even so, it was a little surprising that our method only reached an accuracy of 64.6%, and that it was difficult to move above this accuracy. Our initial thought was that the accuracy would be around 70-80%, especially given that the rbf kernel allows for complex, nonlinear boundaries.

### **ResNet V2 and VGGNet Model**

We learned a lot about the realities of training a machine learning model using PyTorch and how there were many choices in optimizers, schedulers, and even the overall framework to use for the software of the model. We learned to apply methods like saving model checkpoints or model parameters for later retraining or selecting the best performing model out of all the trained epochs that taught us about practical machine learning techniques.

We were surprised by the sheer number of hyperparameters and implementation choices that were available once we took the classification problem out of the theoretical context to apply it to a real problem. While trying to apply feature extraction methods like HOG ourselves, we were surprised by how much the choice of the number of descriptors or the way we choose to build a vocabulary from descriptors could affect our model's latency and accuracy. We learned a lot from the latency and accuracy tradeoff problems in training a classifier. For the SM classifier, we had to choose between building and exporting more features— which increases accuracy but takes more time— or reducing the number of vocabularies and features to improve our training times and make retraining the model feasible without hindering our performance.



For the neural network model, if we had more time we could have explored adding self attention blocks that could encode more of the relationships between patches of the input butterfly image to learn features that could better describe each butterfly and moth class for classification. We also may have tried training even deeper ResNet models, which we decided against in this case due to the constraints of the project and competition deadline. Another way we may have iterated on our model development is running k-folds cross validation to identify the best performing models from the ones we saved during training, and also tuning the hyperparameters such as learning rate or batch size with a grid search to identify the best parameters to use based on test set accuracy.